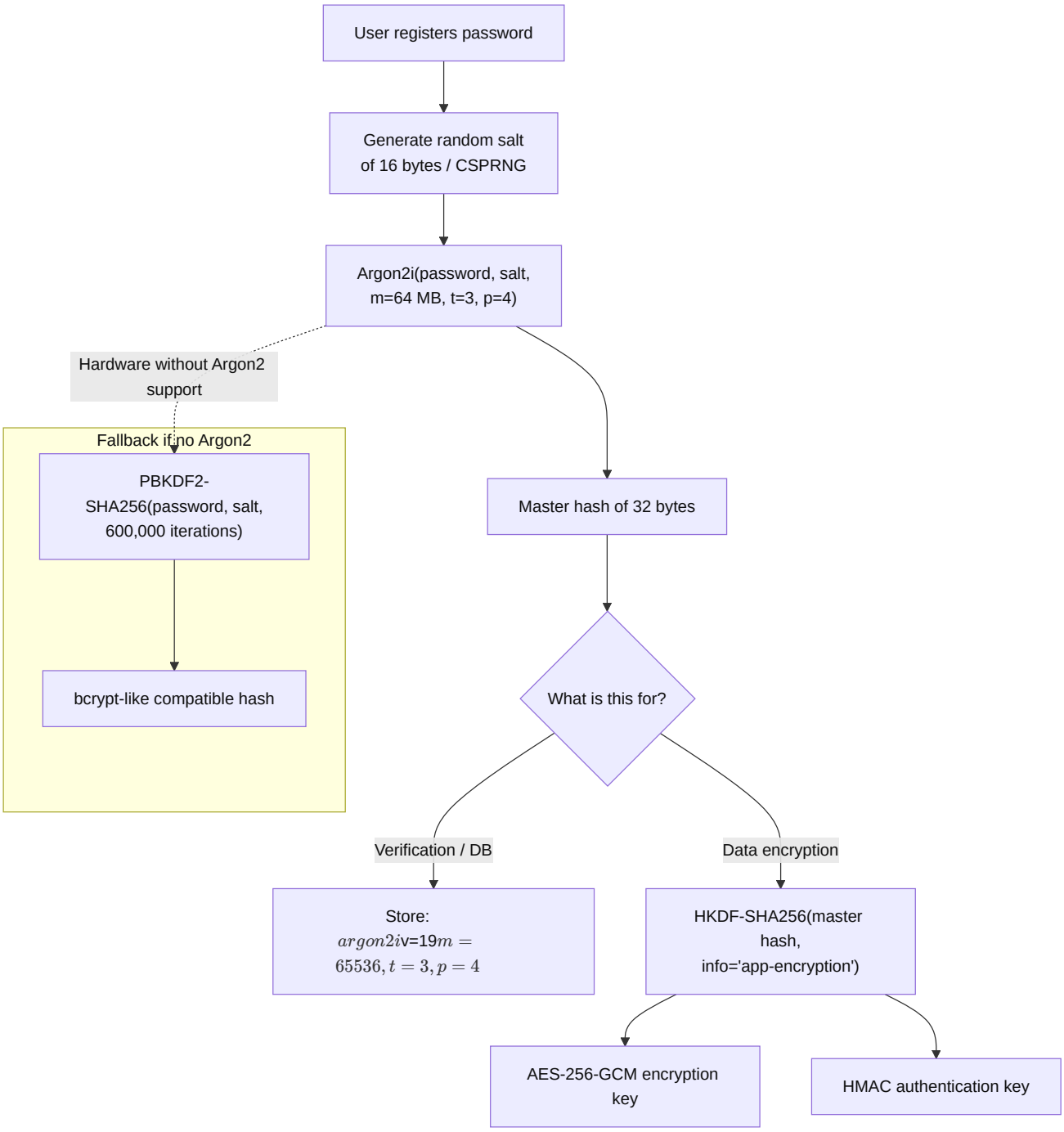
Complex Mermaid in PDF

Purpose: This document tests how Mermaid diagrams are rendered in Leaf. All diagrams below are intentionally complex (nested flows, sequence interactions, decision trees) to exercise edge cases in rendering, layout, and text wrapping.

---

1. Overview of the Process

When a user creates an account, the password is **never stored in plaintext**. Instead, it passes through a three-stage pipeline: random salt generation, memory-hard hashing (Argon2i), and key derivation (HKDF) for specific uses.



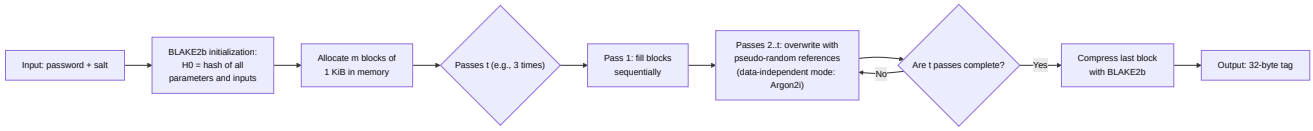
Key points:

- The *salt* ensures that two users with the same password have different hashes.
- Parameters `m` (memory), `t` (iterations), and `p` (parallelism) are saved alongside the hash for later verification.

---

2. Inside Argon2i

Argon2i is the winner of the Password Hashing Competition (2015). Its strength lies in requiring **significant memory**, which makes GPU/ASIC attacks prohibitively expensive.

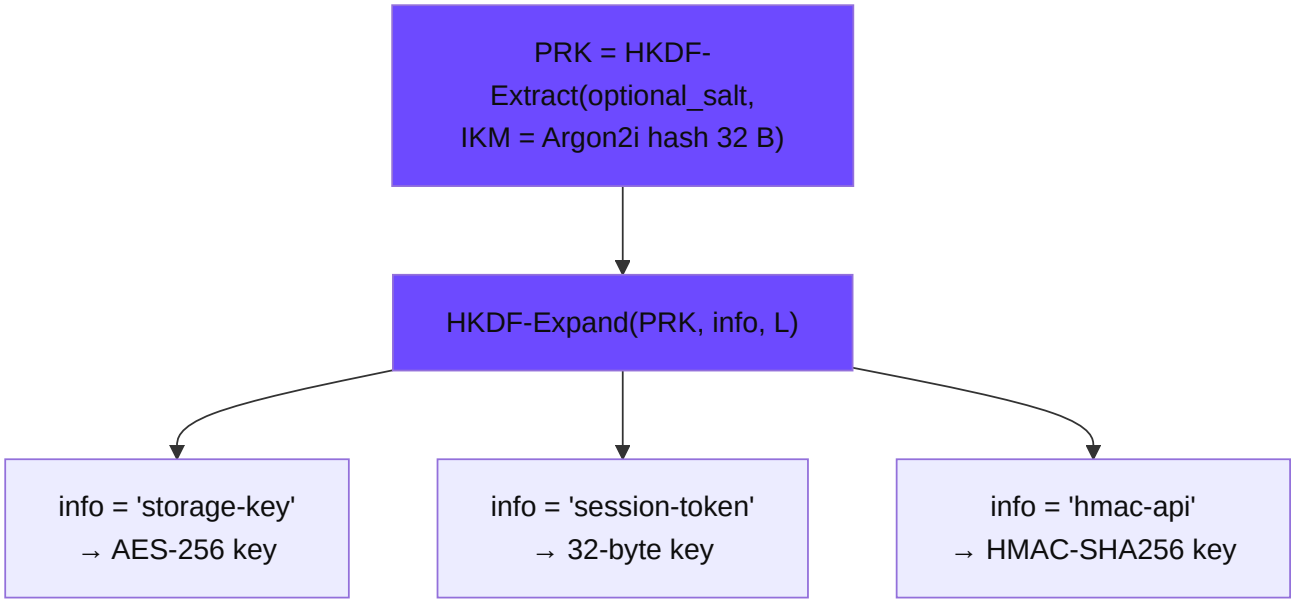


Why mode `i` (data-independent): memory addresses depend only on the salt, not on content, offering greater resistance to time-memory trade-off attacks — ideal for passwords (Argon2d is preferred for proof-of-work mining).

---

3. From Master Hash to Keys: HKDF

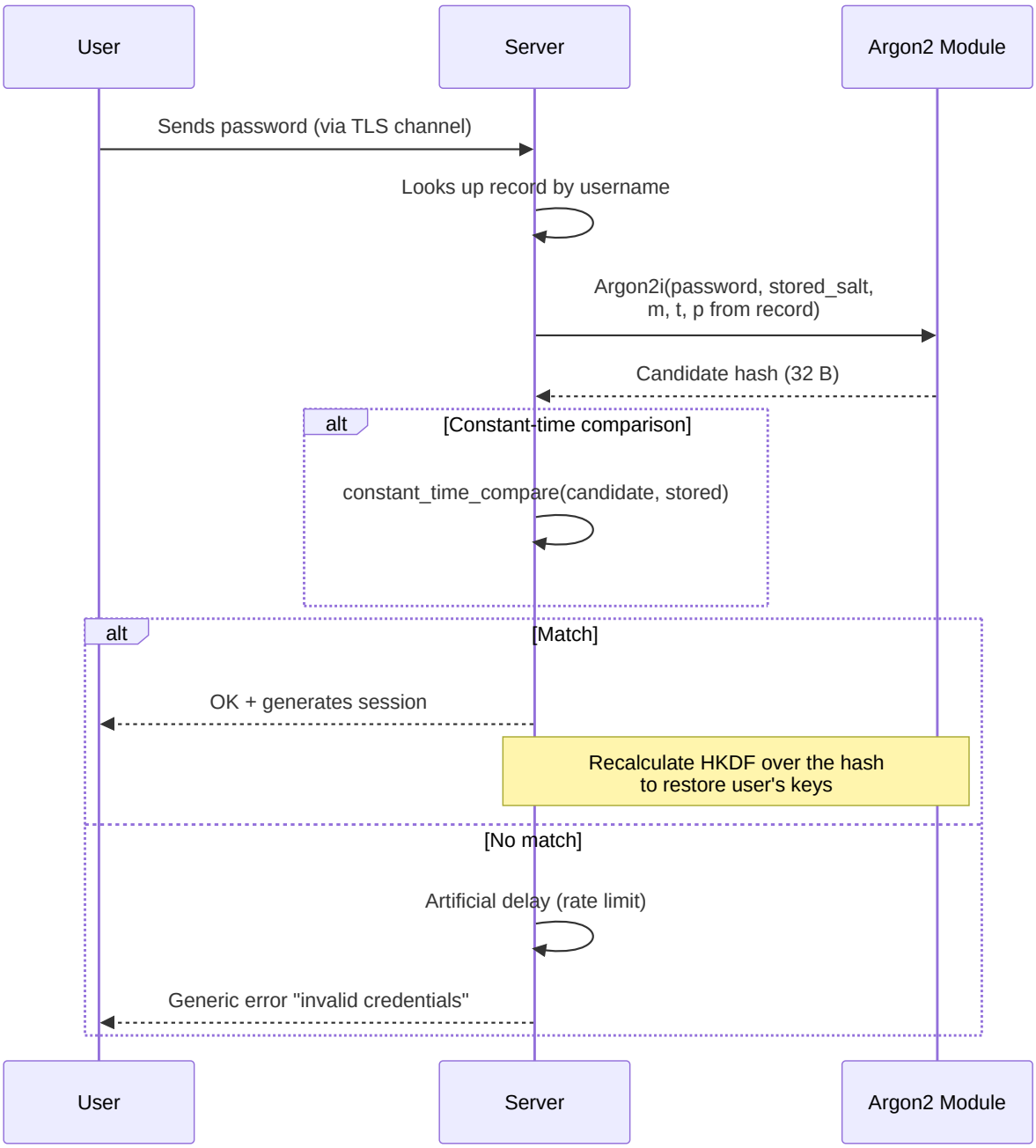
HKDF (RFC 5869) is fast and designed to expand already-random material. Since Argon2i delivers that, we **do not need** another slow function here.



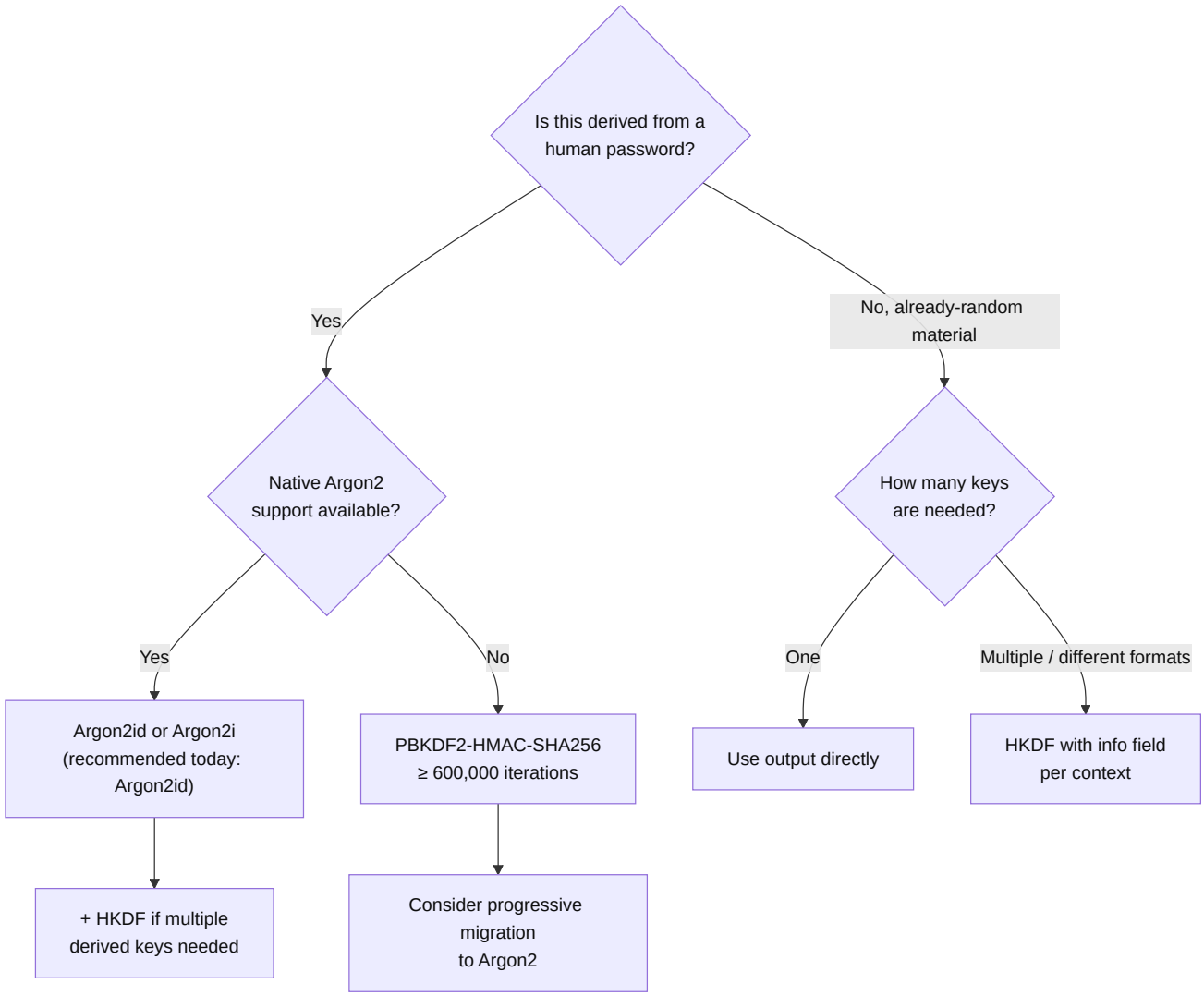
The `info` field acts as *domain separation*: keys derived from the same root secret are cryptographically independent from each other, preventing key reuse issues.

---

4. Verification during Login



5. Decision Tree for Choosing a KDF



6. Recommended Parameter Summary (2024+)

Function	Typical Parameter	Comment
Argon2i/id	m = 64 MiB, t = 3, p = 4	OWASP recommends m ≥ 19 MiB, t ≥ 2
PBKDF2-HMAC-SHA256	600,000 iterations	Current OWASP minimum

HKDF-SHA256	—	Not configurable for cost, linear scaling

> ***Final note:*** *never use plain SHA-256 for passwords. It's fast — exactly what an attacker wants. The functions above exist specifically to be slow in the right direction: annoying the attacker, not the legitimate user.*

Document generated for Mermaid rendering validation in Leaf environment.